StackRoute Ops — Azure Deployment Runbook

Azure App Service deployment with Azure AD (Entra ID) authentication

Prepared for	IT / Infrastructure Team
Prepared by	Divya Kherajani (divya.kherajani@infoglen.com)
Date	2026-07-24
Status	Ready for deployment

This deployment is to be performed by the IT team only. Application/business stakeholders should not need to run any of the steps below themselves — this document exists so IT has everything needed to do it independently, without follow-up questions to the requester except where a decision is explicitly called out below.

---

1. Purpose and scope

This runbook provisions StackRoute Ops (the EdTech Ops Hub) on Azure as two Linux App Services (frontend + backend), backed by Azure Cosmos DB for MongoDB, with sign-in gated by Azure AD (Entra ID) — no separate credentials to manage for end users beyond their existing Microsoft account.

In scope: first production deployment, manual (no CI/CD pipeline yet). **Out of scope:** CI/CD automation, ongoing monitoring/alerting setup, disaster recovery — these can follow as separate work once this baseline is live.

---

2. Application overview

Field	Value
App name	StackRoute Ops / EdTech Ops Hub
Stack	React (frontend), FastAPI/Python (backend), MongoDB (database)
Repository	https://github.com/aryaghai6/EdTech-Ops-Hub
Branch to deploy	main
Container definitions	Dockerfile.frontend, Dockerfile.backend (repository root)

Before starting: confirm with the requester that the main branch is up to date with the latest committed code. If IT pulls the repository and finds pending/uncommitted work referenced elsewhere, check with the requester before proceeding — don't deploy from a stale branch.

3. Information required before starting

Please confirm or supply the following before beginning Step 1. Where a default is suggested, IT may use it unless the organization's naming/tagging standards require something else.

#	Item	Default / suggestion	Who decides
1	Azure subscription to deploy into	—	Requester / IT
2	Azure region	eastus	IT
3	Resource group name	rg-edtech-ops-hub	IT (naming standard)
4	Container Registry name (globally unique)	acredtechopshub	IT
5	Backend App Service name (globally unique)	edtech-ops-backend	IT
6	Frontend App Service name (globally unique)	edtech-ops-frontend	IT
7	Cosmos DB for MongoDB cluster name	edtech-ops-mongo	IT
8	Who can register an Azure AD app / grant admin consent	—	IT (Global/App Admin)
9	Initial local admin password (ADMIN_PASSWORD)	IT-generated, shared via password manager	IT
10	Whose Microsoft account is promoted to Admin first	—	Requester
11	Who gets Contributor on the resource group ongoing	—	Requester / IT

4. Architecture summary

```
Browser
   |
   v
Frontend App Service (nginx + React build)   [Azure AD sign-in required]
   |  proxies /api/* server-side
   v
Backend App Service (FastAPI)                [not directly internet-reachable]
   |
   v
Cosmos DB for MongoDB
```

The frontend and backend are deployed as **two separate App Services** (two different *.azurewebsites.net hostnames). This shapes how the Azure AD gate is wired up, and it's important IT follows this exactly rather than the more "obvious" symmetric setup:

- **Azure AD sign-in (App Service Authentication / "Easy Auth") is enabled only on the frontend App Service**, in redirect mode. A browser loading the page is redirected to Microsoft sign-in if not already authenticated. Once signed in, Azure injects identity headers (X-MS-CLIENT-PRINCIPAL-*) into every request the frontend container receives — including calls to /api/*, because those are proxied server-side by nginx rather than called directly by the browser.
- **The backend App Service does NOT get its own "Require authentication" setting.** Its only caller is nginx, running inside the frontend's container, making a server-to-server call with no browser session attached. Turning on "Require authentication" on the backend would reject every one of those proxied calls outright. Instead, the backend trusts the X-MS-CLIENT-PRINCIPAL-* headers nginx forwards from the already-verified frontend request (already implemented in backend/auth.py).
- **The backend is instead locked down at the network level** (Step 8) so it only accepts traffic from the frontend App Service, since it has no sign-in gate of its own to fall back on.

If a future phase needs the frontend and backend to be fully independent (e.g. a mobile client calling the backend directly), that requires a different auth pattern (MSAL-based token acquisition in each client) — flag that to the requester as separate work rather than improvising it here.

5. Deployment steps

Run these from a machine with Azure CLI installed and authenticated (az login), with the repository checked out locally on the main branch. Placeholders <RG>, <LOCATION>, <ACR_NAME>, <BACKEND_APP>, <FRONTEND_APP>, <COSMOS_CLUSTER> refer to the values agreed in Section 3.

Step 1 — Resource group and Container Registry

```
az group create --name <RG> --location <LOCATION>

az acr create --resource-group <RG> --name <ACR_NAME> --sku Basic --admin-enabled true
```

Step 2 — Build and push the container images

Build from the repository root — both Dockerfiles expect that build context.

```
az acr login --name <ACR_NAME>

docker build -f Dockerfile.backend -t <ACR_NAME>.azurecr.io/edtech-backend:latest .
docker push <ACR_NAME>.azurecr.io/edtech-backend:latest

# REACT_APP_BACKEND_URL must be left EMPTY — the frontend calls its own origin's
# /api path (same-origin, proxied by nginx), never an absolute backend URL.
docker build -f Dockerfile.frontend --build-arg REACT_APP_BACKEND_URL= \
  -t <ACR_NAME>.azurecr.io/edtech-frontend:latest .
docker push <ACR_NAME>.azurecr.io/edtech-frontend:latest
```

Step 3 — Cosmos DB for MongoDB (vCore)

The Azure Portal wizard is the most reliable path for this resource type (CLI/extension support for vCore clusters has moved around across CLI versions — check `az cosmosdb mongocluster --help` first if scripting this is preferred, and confirm the exact flags against the CLI version in use before relying on them).

- Portal → Create a resource → search Azure Cosmos DB → Azure Cosmos DB for MongoDB → choose the **vCore** cluster type (not the older RU-based API — vCore is far closer to real MongoDB wire compatibility, which matters since the app was built and tested against real MongoDB).
- Resource group <RG>, cluster name <COSMOS_CLUSTER>, region <LOCATION>, set an admin username/password, choose the smallest tier that fits (an M25 tier, or the free tier if offered, is sufficient for this app's load), storage autoscale on.
- Networking: for the first deployment, "Allow public access from Azure services and resources within Azure" is the simplest option. Tighten this later (Private Endpoint + VNet integration) as a follow-up hardening task.
- Once created: Connection strings blade → copy the full mongodb+srv://... string exactly as shown. Do not hand-construct or edit it — vCore connection strings carry specific parameters (typically tls=true&authMechanism=SCRAM-SHA-256&retrywrites=false) that are easy to get subtly wrong by retyping.

Retain that connection string for Step 6 (MONGO_URL).

Step 4 — App Service Plan and the two Web Apps

```
az appservice plan create --name asp-edtech-ops-hub --resource-group <RG> \
  --is-linux --sku B1

az webapp create --resource-group <RG> --plan asp-edtech-ops-hub \
  --name <BACKEND_APP> \
  --deployment-container-image-name <ACR_NAME>.azurecr.io/edtech-backend:latest

az webapp create --resource-group <RG> --plan asp-edtech-ops-hub \
  --name <FRONTEND_APP> \
  --deployment-container-image-name <ACR_NAME>.azurecr.io/edtech-frontend:latest
```

Step 5 — Connect each Web App to the registry

```
az acr credential show --name <ACR_NAME>
# note the "username" and one "password" value from the output, then:

az webapp config container set --name <BACKEND_APP> --resource-group <RG> \
  --docker-custom-image-name <ACR_NAME>.azurecr.io/edtech-backend:latest \
  --docker-registry-server-url https://<ACR_NAME>.azurecr.io \
  --docker-registry-server-user <acr-username> \
  --docker-registry-server-password <acr-password>

az webapp config container set --name <FRONTEND_APP> --resource-group <RG> \
  --docker-custom-image-name <ACR_NAME>.azurecr.io/edtech-frontend:latest \
  --docker-registry-server-url https://<ACR_NAME>.azurecr.io \
  --docker-registry-server-user <acr-username> \
  --docker-registry-server-password <acr-password>
```

Step 6 — App settings

Backend:

```
az webapp config appsettings set --name <BACKEND_APP> --resource-group <RG> --settings \
  WEBSITES_PORT=8001 \
  MONGO_URL="<the mongodb+srv connection string from Step 3>" \
  DB_NAME=edtech_ops_production \
  JWT_SECRET="$(openssl rand -hex 32)" \
  ADMIN_USERNAME=admin \
  ADMIN_PASSWORD="<the password agreed in Section 3, item 9>" \
  COOKIE_SECURE=true \
  CORS_ORIGINS="https://<FRONTEND_APP>.azurewebsites.net" \
  ENABLE_API_DOCS=false
```

Frontend:

```
az webapp config appsettings set --name <FRONTEND_APP> --resource-group <RG> --settings \
  WEBSITES_PORT=3000 \
  BACKEND_ORIGIN="https://<BACKEND_APP>.azurewebsites.net"
```

COOKIE_SECURE=true is required — App Service serves everything over HTTPS, and the session cookie's Secure flag must match.

Step 7 — Enable Azure AD sign-in (Easy Auth) — frontend only

Portal → <FRONTEND_APP> App Service → Authentication → Add identity provider:

- Identity provider: **Microsoft**.
- App registration: **Create new app registration** — suggested name edtech-ops-hub. Supported account types: **Current tenant — single tenant**, unless sign-in from other organizations' Azure AD tenants is explicitly required.
- Restrict access: **Require authentication**.
- Unauthenticated requests: **HTTP 302 Found redirect: recommended for websites**.
- Token store: leave enabled (default).
- Save.

> **Do not** repeat this step on <BACKEND_APP>. Per Section 4, enabling "Require authentication" there breaks every request proxied from the frontend.

Step 8 — Restrict the backend to frontend-only traffic

```
az webapp show --name <FRONTEND_APP> --resource-group <RG> \
  --query outboundIpAddresses -o tsv
```

Add each IP from that comma-separated list as an allow rule on the backend (everything else is denied by default once any rule exists):

```
az webapp config access-restriction add --name <BACKEND_APP> --resource-group <RG> \
  --rule-name AllowFrontendOnly --action Allow --priority 100 \
  --ip-address <first-outbound-ip>/32

# repeat with an incrementing --priority for each remaining IP in the list
```

This is the fastest path to a working restriction. The more durable version — VNet-integrating the frontend App Service and giving the backend a Private Endpoint instead of a public IP allow-list — is recommended as a follow-up hardening task, since outboundIpAddresses can grow or change if the plan is ever scaled.

Step 9 — First sign-in and admin promotion

- Browse to https://<FRONTEND_APP>.azurewebsites.net. Confirm you're redirected to Microsoft sign-in.
- Sign in with the Microsoft account identified in Section 3, item 10. It will land in the app with the **Viewer** role (read-only) — every new Azure AD sign-in is auto-provisioned at the lowest privilege level by design; nobody gets elevated access automatically.
- Separately, sign in with the local admin account from Step 6 (ADMIN_USERNAME / ADMIN_PASSWORD) at https://<FRONTEND_APP>.azurewebsites.net/login.
- As that local admin: Settings → User Accounts, find the account from the previous step (its username is the sign-in email), and change its role from **Viewer** to **Admin**.
- Store the local admin credentials securely (password manager / Key Vault) as a break-glass fallback — day-to-day use should be via Microsoft sign-in from here on.

---

6. Validation checklist

[ ] Loading the frontend URL in a private/incognito window redirects to Microsoft sign-in before showing any app content.

[ ] After signing in, the Dashboard loads real data (not a 401) — confirms the /api/* reverse proxy and identity-header forwarding are working.

[ ] https://<BACKEND_APP>.azurewebsites.net/api/meta, hit directly from outside, is refused/times out (confirms Step 8's restriction is active).

[ ] https://<FRONTEND_APP>.azurewebsites.net/.auth/me, while signed in, returns your claims as JSON (confirms Easy Auth is active).

[ ] The account from Section 3, item 10 has been promoted to Admin.

[ ] Settings → Audit Log (as admin) shows sign-ins/actions correctly attributed.

7. Post-deployment handback

Please return the following to the requester once deployment is complete:

Item	Value
Frontend URL	https://<FRONTEND_APP>.azurewebsites.net
Backend URL (internal use only)	https://<BACKEND_APP>.azurewebsites.net
Resource group	<RG>
Local admin username	<ADMIN_USERNAME>
Local admin password	(shared via password manager, not this document)
Requester's account promoted to Admin?	Yes / No
Azure AD app registration name	edtech-ops-hub

8. Troubleshooting

Symptom	Likely cause
Redirect loop on sign-in	COOKIE_SECURE misconfigured, or testing over plain HTTP — Easy Auth requires HTTPS (default on *.azurewebsites.net).
Dashboard loads but API calls return 401 in the browser console	BACKEND_ORIGIN on the frontend app settings doesn't match the backend's real URL, or WEBSITES_PORT is wrong on either app (8001 backend, 3000 frontend).
New Azure AD sign-ins aren't appearing as new users in the app	Confirm Easy Auth is actually injecting headers via .../.auth/me while signed in.
MongoDB connection errors	Re-copy the connection string from the Cosmos DB portal blade rather than editing by hand.

9. Support and contacts

Role	Name	Contact
Business / requester contact	Divya Kherajani	divya.kherajani@infoglen.com
Deploying team	IT / Infrastructure	(fill in)
Escalation	(fill in)	(fill in)